HTML QUICK STYLER

Project Report

Submitted to **Manonmaniam Sundaranar** University in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

NAAN MUDHALVAN COURSE
(Agentic AI Application Developer)

Submitted By

L. AMEENAL YASMIN	**23081021802112003**
A. FATHIMA FASMIN	**23081021802112012**
M. PACKEERA BARVIN	**23081021802112023**
S. RAJAF MYDEEN BEEVI	**23081021802112027**

Under the guidance of

Dr.A. Aaisha Nazleem

MCA., M.Phil.,Ph.D

Assistant Professor,

DEPARTMENT OF COMPUTER SCIENCE

ANNAI HAJIRA WOMEN'S COLLEGE

(AFFILIATED TO MANONMANIAM SUNDARANAR UNIVERSITY)

TIRUNELVELI - 627005

MARCH– 2026

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
1.	**ABSTRACT**	
2.	**INTRODUCTION**	1
3.	**SYSTEM REQUIREMENTS** 3.1 HARDWARE REQUIREMENTS 3.2 SOFTWARE REQUIREMENTS 3.3 CORE TECHNOLOGIES	3
4.	**SYSTEM DESIGN** 4.1 TECHNICAL ARCHITECTURE 4.2 PROJECT STRUCTURE	4
5.	**PROJECT WORKFLOW**	7
6.	**ALGORITHM**	9
7.	**IMPLEMENTATION** 7.1 CODING 7.2 OUTPUT	11
8.	**CONCLUSION**	19

HTML Quick Styler

1.ABSTRACT

The rapid growth of web development has increased the demand for tools that simplify the process of creating modern and visually appealing websites. However, many beginners and non-technical users face difficulties in designing and styling web pages using traditional HTML and CSS methods. To address this problem, this project proposes an AI-based system called HTML Quick-Styler, which automatically generates stylish website templates with minimal user input. The system integrates artificial intelligence with modern web development technologies to produce responsive and aesthetically pleasing web pages. It utilizes the IBM Granite 3.3 Instruct model through the Transformers library to generate structured content and design suggestions. The user interacts with the system through a Gradio-based graphical interface, which allows easy input of titles, descriptions, and design preferences. Based on this information, the system generates HTML templates, color palettes, and structured layouts automatically. The application includes multiple elegant templates such as modern business layouts, portfolio pages, SaaS landing pages, and blog-style designs. It also integrates dynamic color palette generation and image selection using external image sources. These features allow the system to produce visually attractive websites without requiring advanced coding knowledge. The proposed system demonstrates how artificial intelligence can significantly simplify the web design process. By automating layout creation, styling, and template generation, it reduces development time and improves productivity. The project can be useful for students, designers, and developers who want to quickly prototype or generate websites.

Overall, the HTML Quick-Styler system highlights the potential of combining AI models with web technologies to create intelligent tools that assist in modern website development.

2.INTRODUCTION

Web development plays a significant role in today's digital world. Businesses, organizations, educational institutions, and individuals rely on websites to communicate information, promote services, and interact with users across the globe. With the rapid growth of the internet, websites have become one of the most important platforms for sharing information and providing online services. A well-designed website helps organizations improve their online presence, attract potential customers, and deliver services effectively.

Traditionally, developing a website requires knowledge of various web technologies such as HTML, CSS, and JavaScript. Developers must create structured layouts, apply suitable styling, and ensure that the website functions properly on different devices and screen sizes. Designing visually appealing web pages also requires knowledge of design principles such as typography, colour combinations, and layout structures. For beginners and students who are new to web development, this process can be challenging and time-consuming. They often struggle with writing complex styling code and organizing different components of a webpage.

In recent years, artificial intelligence has introduced new possibilities in many technological fields, including software development and web design. AI-powered tools can assist developers by generating code, suggesting design ideas, and automating repetitive tasks. These tools help improve productivity and reduce the time required to build applications. AI models are capable of understanding natural language input from users and converting it into structured outputs such as HTML content or design suggestions.

This project focuses on developing an AI-assisted HTML styling and website generation system called **HTML Quick-Styler**. The primary objective of the system is to simplify the process of designing and generating web pages. The system uses the Transformers library to integrate an advanced language model that can generate HTML structures and design-related content. The model processes user input and produces structured outputs that can help users quickly create website layouts.

To make the system easy to use, the project integrates the **Gradio framework** to create an interactive web interface. Gradio provides a simple way to build web-based interfaces for machine learning applications. Through this interface, users can provide input prompts and

receive generated design suggestions from the AI model in real time. This interactive environment makes the system user-friendly and accessible even for users with minimal technical knowledge.

One of the important features of the HTML Quick-Styler system is the automatic generation of colour palettes and design templates. The application includes several modern templates such as portfolio websites, business landing pages, and SaaS product pages. Each template contains structured sections including navigation menus, hero sections, feature descriptions, portfolios, testimonials, and contact forms. These predefined layouts allow users to quickly generate complete web pages without manually designing every element.

The system also supports modern styling techniques and attractive colour themes to improve the visual quality of the generated websites. By combining AI-generated content with predefined design templates, the system helps users create professional-looking websites in a short period of time. This reduces the effort required in front-end development and allows users to focus more on content and functionality.

The main goal of this project is to make website development easier and more accessible for beginners, students, and developers. By integrating artificial intelligence with web design tools, the system demonstrates how AI can simplify the process of creating modern websites. In the future, the system can be enhanced with more advanced templates, responsive design capabilities, and additional AI features that provide intelligent design recommendations and customization options.

3.SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

1. **Processor** : Intel(R) Core(TM) i3/i5 or higher
2. **Installed** : RAM 8.00 GB (7.89 GB usable)
3. **System type** : 64-bit operating system, x64-based processor

3.2 SOFTWARE REQUIREMENTS

1. **Operating System** : Windows 10
2. **Programming Language** : Python 3.9
3. **Development Environment** : Google Colab
4. **Libraries Used** : Transformers (Hugging Face), Gradio, JSON, Regex(re), Datetime
5. **AI Model** : IBM Granite 3.3-2B Instruct.
6. **Browser** : Google Chrome

3.3 Core Technologies Used

1. Python Flask

Flask is a lightweight web framework used to build backend applications. It handles routing, authentication logic, and server-side processing.

2. HTML & CSS

HTML and CSS are used to create the user interface and design the web pages.

3. JavaScript

JavaScript is used to add interactive features and improve user experience.

4. Geolocation APIs

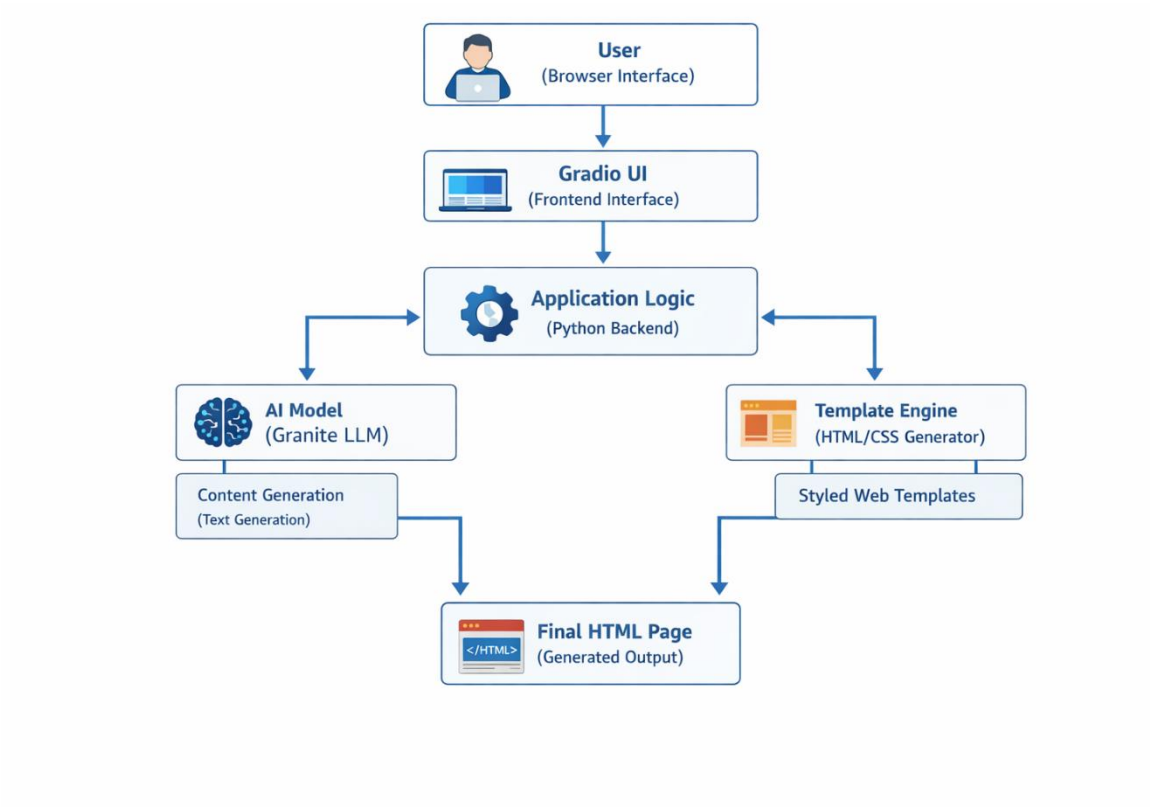
These APIs help detect the user's location using IP address.

5. Database System

The database stores user credentials, login records, and system logs.

4.SYSTEM DESIGN

4.1 TECHNICAL ARCHITECTURE



Project Workflow – Explanation

The system architecture of this project is designed to generate styled HTML webpages automatically using AI and predefined templates.

1. User Layer

The user interacts with the system through a browser interface. The user provides inputs such as website title, description, and design preferences.

2. Frontend Layer (Gradio UI)

The Gradio interface acts as the frontend of the application. It collects user inputs and sends them to the backend processing system. It also displays the generated output to the user.

3. Backend Application Layer

The backend is implemented using Python. This layer processes user input, controls the workflow, and communicates with different modules such as AI model processing and template generation.

4. AI Processing Layer

The project uses the Granite Large Language Model from the Transformers library.

This model helps generate text content such as headings, descriptions, or other webpage content based on user input.

5. Template Generation Layer

This module generates the HTML and CSS structure of the webpage using predefined elegant templates such as:

- Midnight Gold
- Nordic Frost
- Emerald Tech
- Rose Editorial

These templates define the layout, color scheme, fonts, and design style.

6. Output Layer

After processing, the system generates a fully styled HTML webpage.

This output can be previewed directly in the interface and used as a ready-to-deploy website.

4.2 PROJECT STRUCTURE

The project structure describes how the application is organized to perform the website generation process. Even though the project is simple, it is logically divided into different functional parts inside the main file.

```
HTML Quick Styler Project
│
└── Main Application File
 │
 ├── User Interface Section
 ├── Input Processing Section
 ├── HTML Generation Section
 ├── CSS Styling Section
 └── Output Display Section
```

Description:

• **User Interface Section** – Handles the interaction between the user and the system using Gradio.
• **Input Processing Section** – Processes the data entered by the user such as title, description, and images.
• **HTML Generation Section** – Generates the HTML structure of the web page dynamically.
• **CSS Styling Section** – Applies styles and layouts to make the web page visually attractive.
• **Output Display Section** – Displays the final generated HTML page and preview to the user.

5.PROJECT WORKFLOW

The implementation of the **HTML Quick Styler** project was carried out through several stages such as environment setup, development of the styling system, interface creation, and application testing. These steps ensure that the system works properly and generates styled HTML webpages based on the user's input.

Step 1: Setting Up the Development Environment

The first step is preparing the development environment by installing the required Python libraries. The **Gradio framework** is mainly used to create the interactive interface of the application.

Command:

pip install transformers

pip install gradio

Step 2: Importing Required Libraries

After installing the libraries, the necessary Python modules such as **Gradio, OS, Random, JSON, and other supporting libraries** are imported into the program. These libraries help in creating the interface, managing files, and generating webpage styles.

Step 3: Implementing the Styling System

A styling system is developed to generate visually appealing webpages. The system includes different **colour palettes and design themes** that automatically apply styles to the generated webpage layout.

Step 4: Developing the User Interface

The application interface is created using the **Gradio framework**. It provides input fields where users can enter the webpage title, description, and choose a preferred design style. The interface also supports image upload and webpage generation options.

Step 5: Generating HTML and CSS Code

The system processes the user input and automatically generates the required **HTML structure and CSS styling**. The generated code represents the layout and design of the webpage.

Step 6: Running and Testing the Application

Finally, the application is executed and tested to verify that all features such as **webpage generation, styling, preview display, and HTML file download** work correctly. The system ensures that users can easily create and export styled webpages.

6.ALGORITHM

1. Start & Open Application

• Launch the **HTML Quick-Styler application** in a web browser.
• Load the main interface with input panel and output panel.

2. Enter Page Details

• User enters **Page Title** and **Page Description / Sections**.
• User selects the **Color Style** for the webpage.
• User optionally change their page theme dark or light
• Validate all inputs before proceeding.

3. Generate HTML Page

• User clicks the **Generate Page** button.
• System processes the input details.
• AI generates the **HTML and CSS code** for the webpage.

4. Display Generated Code

• The system displays the **generated HTML/CSS code** in the output panel.
• User can review the generated code.

5. Live Preview Webpage

• System renders the **generated webpage preview**.
• User checks the design, layout, and content.

6. Code explanation

• User opens the **code explanation**
• System analyzes the generated webpage.
• Provide **codes explanation**

7. Download HTML File

• User clicks the **Download HTML** option.

• System saves the generated webpage as an **HTML file** on the user's device.

8. User Guide (Optional)

• User can open the **Guide section**.

• System displays instructions on how to use the application.

9. End

• User exits the application or closes the browser.

7.IMPLEMENTATION

7.1 CODING:

1. Model Initialization

```
from transformers import pipeline
pipe = pipeline("text-generation", model="ibm-granite/granite-3.3-2b-instruct")
messages = [{"role": "user", "content": "Who are you?"}]
pipe(messages)
```

2. Color Palette Generator

```
class ColorPaletteGenerator:
    @staticmethod
    def generate_palette(style: str = "modern") -> Dict[str, str]:
        palettes = {
            "modern":   {"primary": "#667eea", "secondary": "#764ba2", "accent": "#f093fb",
                        "background": "#0f172a", "surface": "#1e293b",
                        "text_primary": "#f1f5f9", "text_secondary": "#cbd5e1", "border":
"#334155"},
            "vibrant":  {"primary": "#ff6b6b", "secondary": "#ff8e72", "accent": "#f5576c",
                        "background": "#0f172a", "surface": "#1e293b",
                        "text_primary": "#f1f5f9", "text_secondary": "#cbd5e1", "border":
"#334155"},
            "ocean":    {"primary": "#0099cc", "secondary": "#006699", "accent": "#00ccff",
                        "background": "#0f172a", "surface": "#1e293b",
                        "text_primary": "#f1f5f9", "text_secondary": "#cbd5e1", "border":
"#334155"},
        }
        return palettes.get(style.lower(), palettes["modern"])
```

3. HTML Page Builder — Section Keyword Extractor

```
class HTMLPageBuilder:
    @staticmethod
    def extract_sections(description: str) -> List[str]:
        sections = []
        section_keywords = {
            "hero":         ["hero", "banner", "landing"],
            "features":     ["features", "services", "benefits"],
            "about":        ["about", "team", "company"],
            "portfolio":    ["portfolio", "gallery", "work", "projects"],
            "testimonials": ["testimonial", "reviews", "feedback"],
            "pricing":      ["pricing", "plans", "cost"],
            "contact":      ["contact", "form", "reach"],
            "footer":       ["footer", "bottom"],
        }
        desc_lower = description.lower()
        for section_type, keywords in section_keywords.items():
            for kw in keywords:
                if kw in desc_lower:
                    sections.append(section_type)
                    break
        if not sections:
            sections = ["hero", "features", "contact", "footer"]
        return sections
```

4. CSS Generator

```
def generate_css(palette):
    css = f"""
    body {{
        background: {palette['primary']};
        color: white;
        font-family: Arial, sans-serif;
```

```
    }}

    header {{
        background: {palette['secondary']};
        padding: 20px;
        text-align: center;
    }}

    section {{
        padding: 40px;
    }}
    """
    return css
```

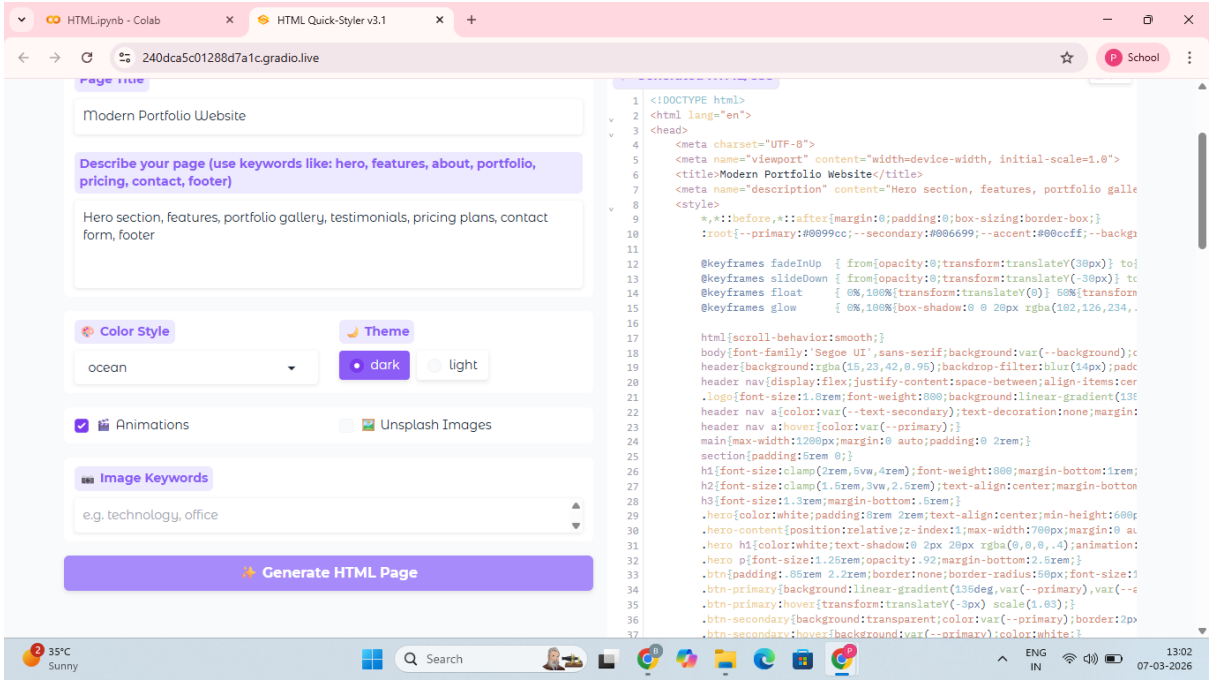
5.HTML template generator

```
def generate_html(style):
colors = generate_palette(style)

html = f"""
<html>
<head>
<style>
body {{
background-color: {colors[2]};
font-family: Arial, sans-serif;
margin: 0;
padding: 0;
}}

header {{
background-color: {colors[0]};
color: white;
padding: 20px;
```

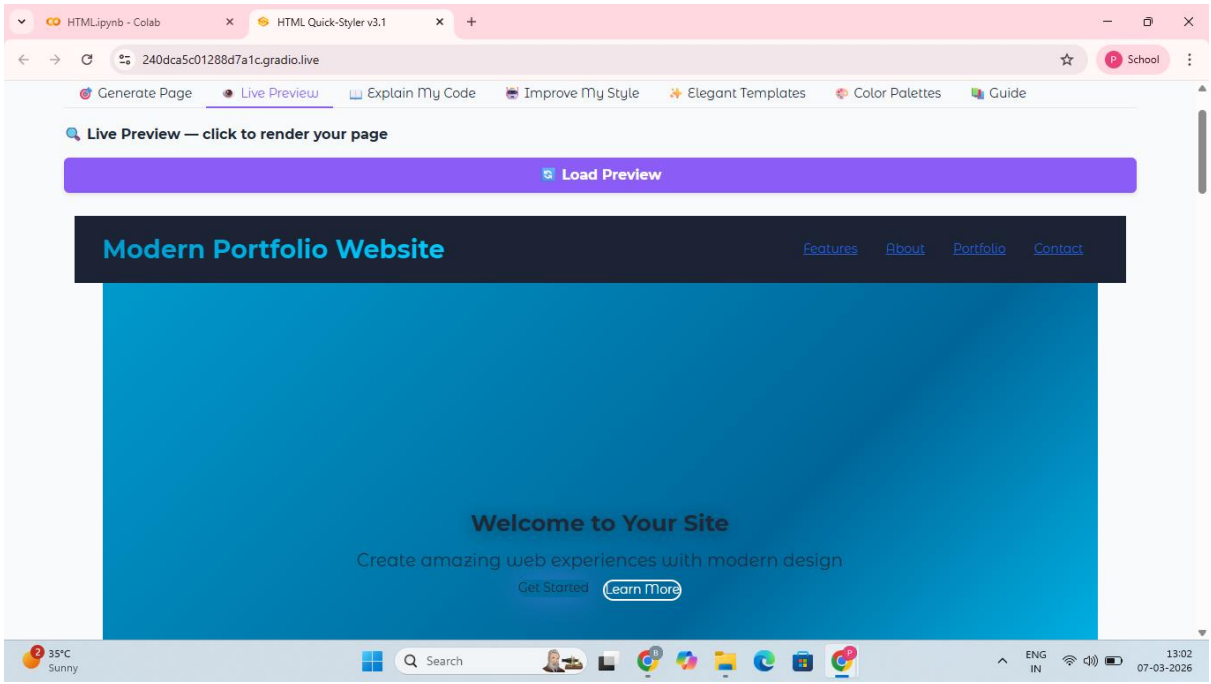
```
text-align: center;
}}

section {{
padding: 40px;
text-align: center;
}}
button {{
background-color: {colors[1]};
border: none;
padding: 10px 20px;
color: white;
font-size: 16px;
cursor: pointer;
}}
</style>
</head>
<body>
<header>
<h1>HTML Quick Styler</h1>
<p>AI Generated Web Page</p>
</header>
<section>
<h2>Welcome to the Demo Page</h2>
<p>This page was generated using the HTML Quick Styler tool.</p>
<button>Learn More</button>
</section>
</body>
</html>
"""
return html
```

7.2 OUTPUT:

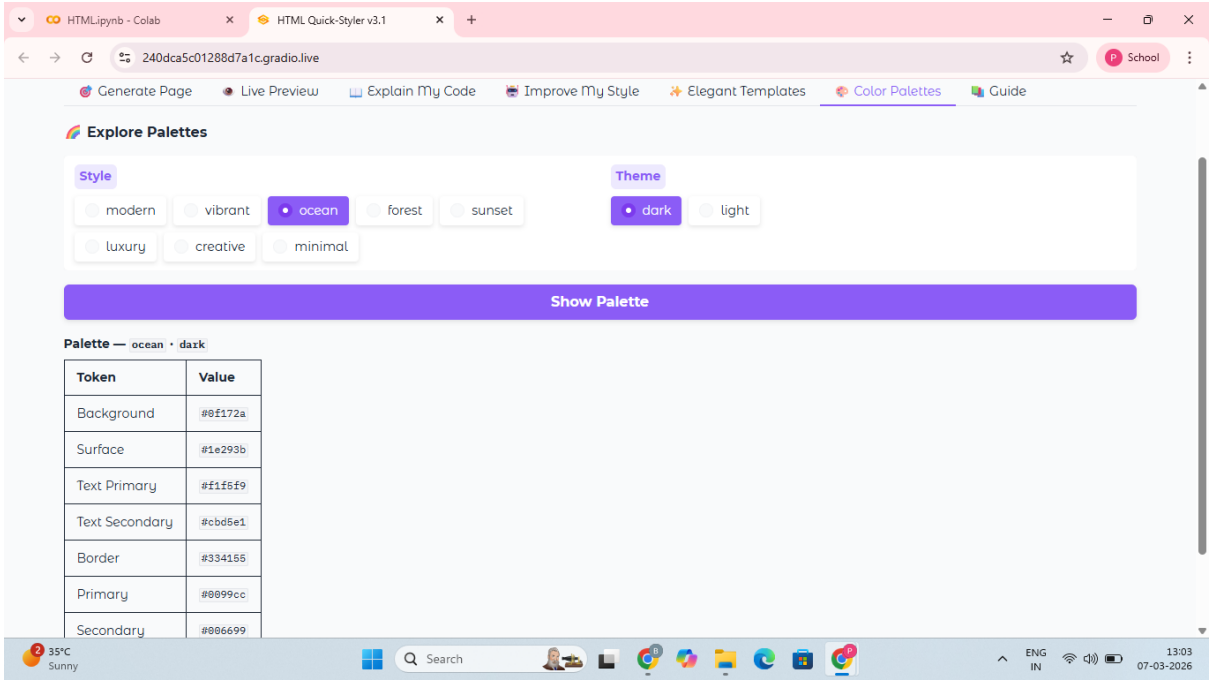
1.HOME PAGE



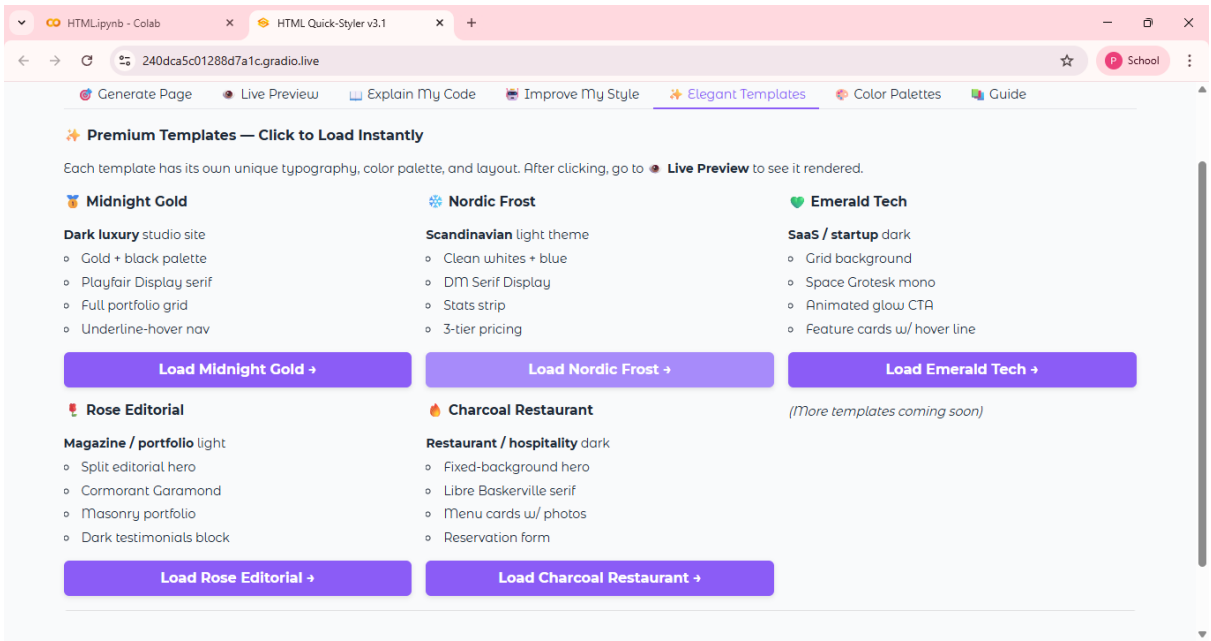
2. LIVE WEBPAGE PREVIEW



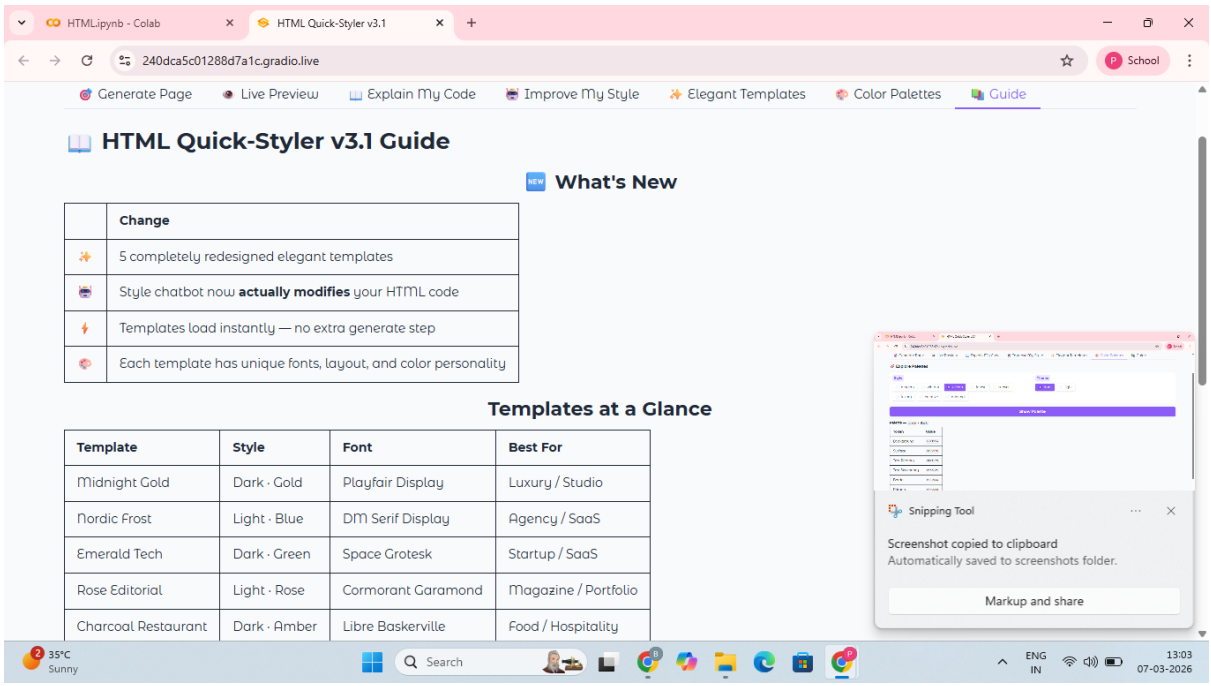
3. COLOR PALETTE PAGE



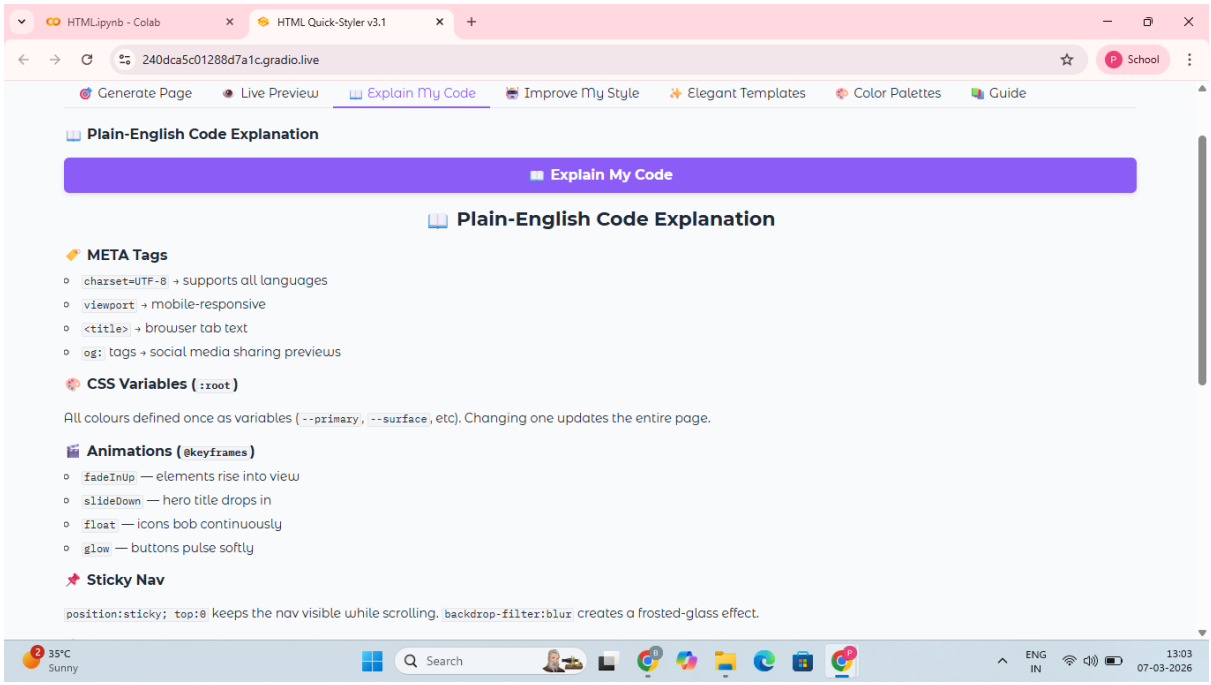
4. TEMPLATE SELECTION PAGE



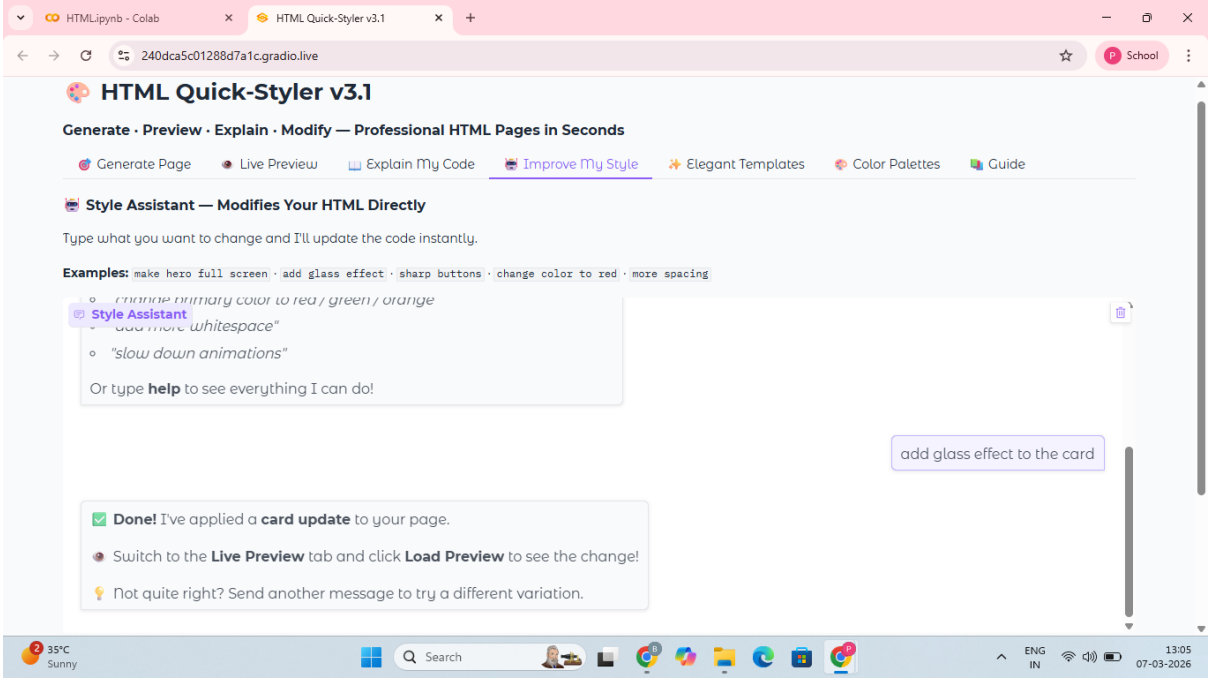
5. USER GUIDE PAGE



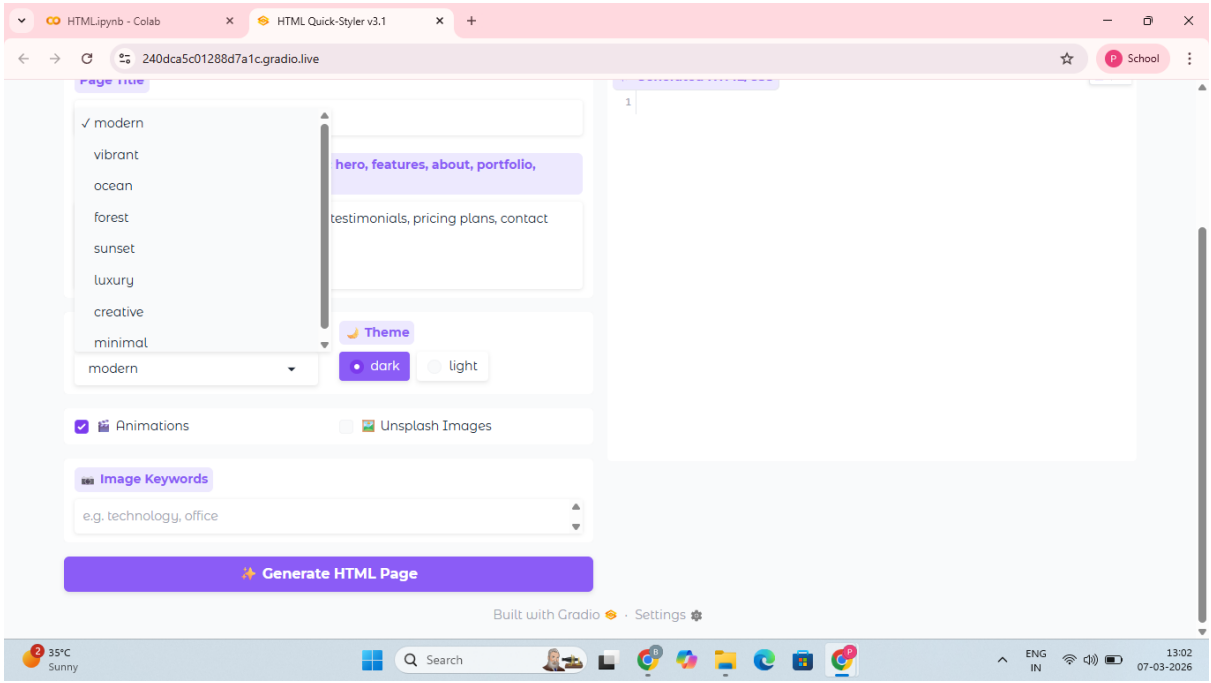
6. Code explanation page



7.Improve Style page



8.Change color and theme



8.CONCLUSION

This project presented the development of an AI-assisted web design tool called HTML Quick-Styler, which automates the generation of modern website templates. The system demonstrates how artificial intelligence can be integrated with web technologies to simplify the website development process. By using the Transformers library and the IBM Granite language model, the system is capable of generating structured web layouts and design elements based on user input. The Gradio interface further improves accessibility by allowing users to interact with the system through a simple graphical environment. This combination of AI and interactive design makes the tool easy to use even for individuals with limited programming knowledge. The project also introduced several useful features such as automatic template generation, dynamic color palette creation, and image selection based on keywords. These features help users create visually attractive web pages quickly without manually writing extensive HTML or CSS code. Experimental analysis showed that the system successfully generates structured and responsive templates that can be directly used for website development. The generated designs include common sections required in modern websites, making them suitable for portfolios, business pages, and product landing pages. In the future, the system can be improved by adding more templates, integrating advanced AI design models, and supporting automatic CSS customization based on user preferences. Additional features such as drag-and-drop editing and real-time preview could further enhance usability. Overall, the HTML Quick-Styler system demonstrates the potential of AI-driven tools in modern web development and highlights how automation can make website design faster, easier, and more accessible.